

Complex diagram examples

This document pushes Lout's `diag` package harder than `diag_gallery.md`: a real grammar in railroad-diagram form, a binary search tree, a flowchart with mixed arrow styles, and a single diagram that combines several constructs at once. Every figure is emitted via the 'lout raw passthrough fence'; `mdlout` auto-detects the `@Diag/@Tree/@SyntaxDiag` macros and pulls in the right `@SysInclude` lines.

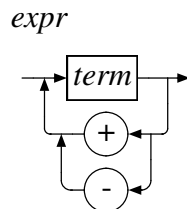
A grammar for arithmetic expressions

The classic four-rule expression grammar:

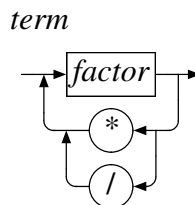
`expr = term ( ( "+" | "-" ) term )* ; term = factor ( ( "" | "/" ) factor )* ; factor = number | identifier | "(" expr ")" ; number = digit+ ;`

Below each rule is rendered as a railroad-style `@SyntaxDiag`. Sequencing uses `@Sequence`, alternatives use `@Select`, and the trailing `*` uses `@Loop` so the diagram visibly returns to the front.

`expr`



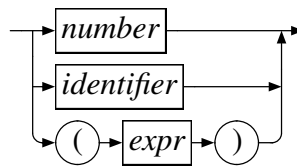
`term`



`factor`

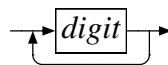
factor

- 2 -



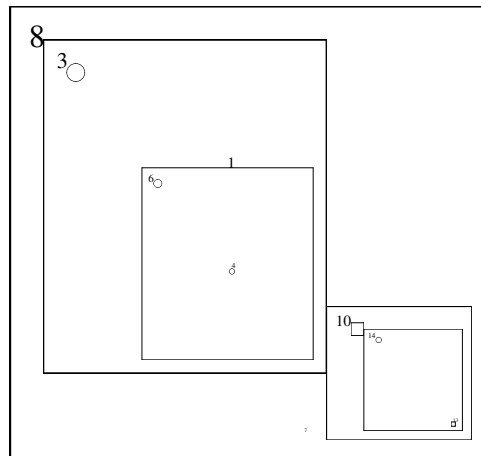
number

number



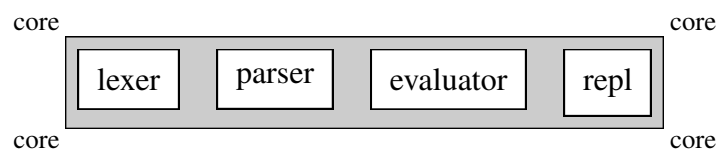
A binary search tree

A small BST holding the values { 8, 3, 10, 1, 6, 14, 4, 7, 13 } after in-order insertion. Note how @Tree nests three deep at the leaves and how @Circle is used for the leaves to distinguish them from internal nodes.



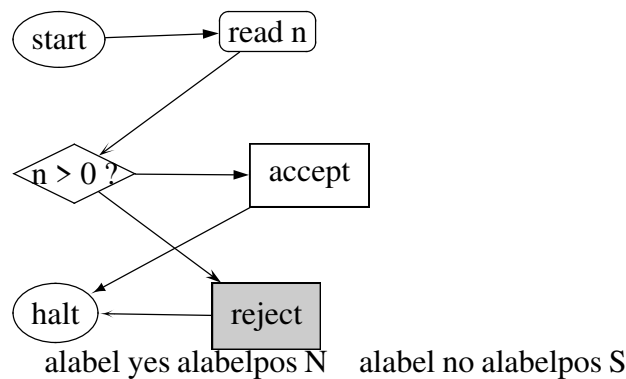
A grey-painted box of labelled subsystems

The `paint` option on @Box (and its variants) fills the node background. Below, a parent @Box `paint { lightgrey }` contains four labelled child nodes representing modules of a small interpreter. The whole thing is the nodelabel'd group `core`.



Flowchart with multiple distinct arrow styles

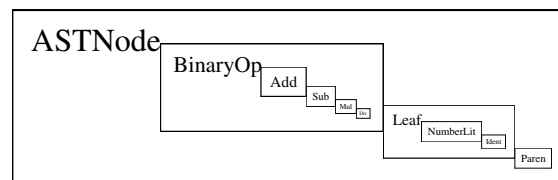
A single @Diag containing five labelled nodes and five links, each link drawn with a different arrowstyle to verify that arrowhead variants coexist correctly in one diagram. The shapes (@Diamond for a decision, @CurveBox for I/O, @Box for processing) follow standard ANSI/ISO flowchart conventions.



Composite: a class diagram with one syntax diagram inside

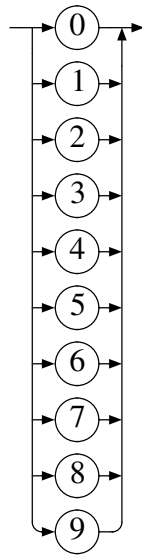
A single page can mix @Diag with embedded @SyntaxDiag figures. Here a class-hierarchy @Tree describes the AST of the arithmetic grammar above, and a small @SyntaxDiag appears alongside it as a legend fragment for the digit terminal.

AST hierarchy



digit (terminal legend)

digit



End

Everything above is raw Lout passthrough; the markdown surrounding it is ordinary prose.
If the build succeeded, the rendered HTML/PDF will show ten distinct figures.